Rapport de Projet : Agent Bibliothécaire de l'Humeur

Le Rôle de l'Agent IA : Cette agent a pour rôle de recommander un livre, un manga ou une autre œuvre écrire (dans les prochaines versions). L'utilisateur lui envoie un texte exposant dans quel **mood** il se trouve permettant ainsi à l'agent de repérer les mots clés et recommander un œuvre pour aider l'utilisateur à se sentir mieux ou simplement nourrir son émotion actuelle.

Déploiement et Architecture

L'application est déployée sous la forme d'un service API FastAPI conteneurisé via Docker sur Hugging Face Spaces. Pour pallier les restrictions d'iframe de l'interface Hugging Face bloquant l'accès au Swagger, nous avons implémenté une route racine (/) renvoyant une interface HTML avec un lien relatif pointant vers la documentation interactive (/docs).

L'URL publique pour tester le projet est : https://gildark012-manga-recommendation-agent.hf.space/docs#/

1. Sûreté (Safety & Guardrails)

Afin de garantir un environnement sécurisé pour nos utilisateurs et d'éviter toute génération de contenu préjudiciable, nous avons mis en place une logique de validation des entrées (*Input validation*). L'agent intègre un filtre de sécurité qui intercepte la requête utilisateur avant tout appel au LLM. Si des termes liés à la dépression profonde ou à l'automutilation sont détectés, l'API court-circuite le processus métier pour renvoyer immédiatement un message bienveillant encourageant la consultation de professionnels de santé. Cette approche protège le système des dérives et assure une éthique *by-design*.

2. Évaluation (Eval)

L'évaluation de la qualité des réponses est automatisée via une fonction de scoring interne déterministe (exact-match), exécutée post-génération. La métrique eval_quality_score analyse la structure de la réponse finale du LLM pour vérifier la présence de deux éléments cruciaux définis au cahier des charges :

- **Les sources :** La présence d'un lien hypertexte (vérification de l'utilisation réussie de l'outil de recherche web).
- **Le raisonnement :** La présence d'une justification textuelle (détection sémantique de conjonctions de causalité comme "car" ou "parce que").

Ce score sur 1.0 est loggé à chaque appel, permettant de s'assurer que l'agent n'a pas halluciné sa réponse sans utiliser ses outils.

3. Observabilité (Cost & Latency)

La traçabilité et l'observabilité sont nativement intégrées dans le payload de réponse de notre API. Le système utilise un modèle LLM optimisé pour minimiser les coûts. À chaque itération de la boucle agentique, notre handler calcule et renvoie les métriques suivantes :

- La latence globale de la requête (en secondes).
- La consommation totale en tokens (input + output cumulant tous les tours de raisonnement).
- Une estimation du coût en dollars (estimated_cost_usd).

- Les outils utilisés étant totalement gratuits et sans clé d'API (OpenLibrary et DuckDuckGo), le coût d'infrastructure externe est nul.

4. Stratégie Agentique (Outils et Boucle ReAct)

Contrairement à un simple LLM "one-shot", notre système implémente une véritable boucle ReAct (*Reasoning + Acting*) plafonnée à 5 étapes maximum pour éviter les boucles infinies. Le *System Prompt* contraint l'agent à chaîner séquentiellement ses actions : il doit d'abord appeler l'outil littéraire (search_literature), observer le résultat, puis utiliser le titre obtenu pour appeler l'outil web (search_web_links) afin de fournir un lien de lecture concret.